

CMSC 161 Term Project Report

Name: Ralph Philip M. Caoile

Student Number: 202*-06***

1. Description of the Scene

The project implements an interactive 3D “**Cat Play Area**” rendered entirely in WebGL 2.0. The environment shows a cozy indoor cat play area containing several objects.

The scene features a munchkin cat named “**Ethel**” which is also known as the meme cat “**Oiia Oiia Cat**”, designed with hierarchical body parts including a cylindrical body, spherical head, cube-based ears, cylindrical tail, short legs, and paw pads. There is also a multi-level cat tree structure composed of sisal-rope-wrapped vertical posts, multiple wooden platforms, and a base platform. Additionally, there are objects that cat used such as, food and water bowl, two toy mice, and of course a box.

The project supports real-time interaction through:

- Mouse orbit, pan, and zoom controls
- Sliders for camera position and lighting parameters
- Cat position controls
- Animation toggles
- Dynamic shadow enable/disable functionality

2. Implementation of Required Topics

2.1 2D Geometry (Primitives, Buffers, and Basic Interaction)

All shapes in the scene is generated in `utils/objects.js`, including cubes, spheres, cylinders, tubes, and the ground plane. Vertex attributes such as positions, normals, texture coordinates, and indices are stored through standard WebGL buffer operations (`gl.createBuffer`, `gl.bindBuffer`, and `gl.bufferData`).

Several objects reuse the same geometry data, such as the cat’s legs and paw pads, as well as the toy mice. User interaction is implemented in `utils/controls.js`, supporting mouse-based orbiting, panning, zooming, and click-based animation toggling.

2.2 3D Objects & Viewing and Transformations & Homogeneous Coordinates

The complete scene is maintained in `utils/scene.js`. The cat is implemented using hierarchical modeling inspired by lecture on articulated objects. The `catRoot` node stores world transformations while child nodes define local transformations for body parts such as the head, tail, ears, and legs.

Transformation matrices are composed using `glmMatrix.mat4` operations including translation, rotation, and scaling. The final model matrices are computed through parent-child matrix multiplication. The cat tree platforms and posts are also the same, using modeling matrices within the standard modeling-viewing-projection pipeline.

2.3 Viewing & Projection

The project uses perspective projection through `mat4.perspective`. The view matrix is generated using `mat4.lookAt` from eye, center, and up vectors. Both the camera and light source use independent view and projection matrices. User sliders allow manipulation of viewing parameters, extending the concepts introduced in Exercises 6 and 7.

2.4 Lighting and Shading (Phong Model)

The shaders implemented in `utils/main.js` use a complete per-fragment Phong reflection model.

The vertex shader computes:

- World-space positions
- Transformed normals using `uNormalMatrix`
- Texture coordinates

The fragment shader implements:

- Ambient lighting
- Diffuse lighting using the $N \cdot L$ term
- Specular lighting using the reflection vector and shininess exponent

Separate material and light properties are maintained for diffuse and specular components. Lighting calculations are based on the concepts introduced in Exercises 8 and 9. The project uses a movable point light source similar to the last exercises.

2.5 Texture Mapping (on at least two surfaces)

The scene applies multiple textures through `utils/textures.js` and object-specific materials:

- `cattree_platform.jpg` for wooden platforms
- `sisal_rope.jpg` for vertical posts
- `fur.jpg` and `tabby.jpg` for the cat body and tail
- `metal1.jpg` for metallic bowl surfaces
- `tiles.jpg` for the floor
- `box1.jpg` for the cardboard box

Texture coordinates are procedurally generated according to object type using cylindrical, planar, spherical, or box mapping techniques. Textures are sampled in the fragment shader using `sampler2D` uniforms and are combined with lighting calculations to modulate the diffuse component.

2.6 Shadows (Shadow Mapping)

The project implements full two-pass shadow mapping based on the algorithm discussed in Lecture 8.

Pass 1: Shadow Pass

First, the scene is rendered from the light source's point of view. This allows the program to determine which parts of the scene are visible to the light and which parts are blocked.

Pass 2: Main Render Pass

Next, the scene is rendered normally from the camera's view. The program compares the current object positions with the information gathered from the light's view to determine whether an area should appear in shadow.

To improve shadow quality, small adjustments are applied to reduce visual errors and unwanted artifacts. Shadows also update automatically whenever the light source moves, and users can enable or disable shadows during runtime.

3. Key Design Decisions and Challenges

Inspirations from previous exercises

Several components evolved from previous CMSC 161 exercises and lecture materials:

- Shader compilation utilities were adapted from Exercises 6–9.
- Phong lighting calculations and normal matrix handling were extended from Exercise 8–9.
- Camera controls and slider interfaces were inspired by Exercises 6 and 7.
- Hierarchical modeling concepts followed the structure introduced in Lecture 10.0.
- Shadow mapping implementation was based directly on the Week 8 shadow lecture materials.

Challenges Encountered and Solutions

A major challenge encountered during development was improving the quality and accuracy of the shadows. Small visual errors sometimes appeared on object surfaces, so the shadow settings had to be carefully adjusted to reduce these artifacts. Another challenge was handling the movable light source. Since the light can change position during runtime, the shadow calculations needed to be updated every frame to ensure that shadows remained correctly aligned with the objects in the scene.